👋 Welcome to the ISKCON Srisailam Online Portal!

Welcome to the team! This document will help you understand the architecture, codebase, and workflows of the project.

🏛️ Project Overview

This represents the digital presence of ISKCON Srisailam. It's a full-stack web application that allows:

- **Devotees** to view events, make donations, and contact the temple.
- **Admins** to manage events and view donation history.
- **System** to send automated emails for new events and donation receipts.

🛠️ Tech Stack (The "MERN" Stack + Extras)

Layer	Technology	Purpose
Frontend	**React.js**	The User Interface.
	Material UI (MUI)	Styling and responsive components.
	Axios	For making HTTP requests to our backend.
Backend	**Node.js & Express**	The API server.
Database	**MongoDB**	NoSQL database storing Users, Events, and Donations.
Email	**Brevo (formerly Sendinblue)**	Transactional email service (Event alerts, Receipts).
Auth	**JWT (JSON Web Tokens)**	Secure authentication for login sessions.

🏗️ Architecture & Key Flows

1. The Backend (`/backend`)

The backend is a lightweight Express server found mainly in `server.js`.

- **Entry Point**: `server.js` initializes the app, connects to MongoDB, and defines routes.
- **Data Models**:
  - `User`: Stores email, password hash, and role ('user' or 'admin').
  - `Event`: Stores title, date, time, image URL, and description.
  - `Donation`: Stores donor details, amount, and PAN info.
- **Key Services**:
  - `brevoEmail.js`: A dedicated module for sending emails. It handles error logging and connection to the Brevo API.

2. The Frontend (`/frontend`)

A React application created with Create React App.

- **Routing**: `App.js` manages navigation. It uses `ProtectedRoute` to lock down pages (like Admin Dashboard) based on login status and roles.
- **State**: We use React's built-in `useState` and `useEffect` hooks. Authentication state (token) is stored in `localStorage`.
- **Design**: We use Material UI's `ThemeProvider` in `App.js` to maintain consistent branding (Saffron & Green).

🚦 Critical Workflows

🛡️ Authentication

1. User signs up -> Backend hashes password (bcrypt) -> Saves to MongoDB.
2. User logs in -> Backend verifies hash -> Issues a **JWT**.
3. Frontend saves JWT. All future requests include this token in the `Authorization` header.

📅 Event System (The "Broadcast" Feature)

When an **Admin** posts a new event:

1. The `POST /events` route is called.
2. The Event is saved to MongoDB.
3. **Magic Step**: The backend immediately queries *all* 'user' emails and triggers `brevoEmail.js` to send a broadcast email announcing the event.

💰 Donation System

1. User fills the form on `DonatePage`.
2. Frontend sends data to `POST /donations`.
3. Backend saves the record.
4. Backend triggers `brevoEmail.js` to send a personalized "Thank You" receipt to the donor.

👷 Your First Day Checklist

1. **Setup Environment**:
   - Install Node.js.
   - Clone the repo.
   - Ask the team for the `.env` file (contains API keys and Database URI).
2. **Explore the Code**:
   - Check `backend/server.js` to see the API routes.
   - Check `frontend/src/App.js` to see how pages are wired together.
   - Look at `backend/brevoEmail.js` to understand how we handle external APIs.
3. **Run Locally**:
   - **Terminal 1 (Backend)**: `cd backend && npm start` (Runs on port 5001 usually).
   - **Terminal 2 (Frontend)**: `cd frontend && npm start` (Runs on port 3000).

⚠️ Important Notes

- **Images**: Currently, we store image *URLs* (strings), not the files themselves. You'll need to host images elsewhere (like Imgur or a cloud bucket) and paste the link.
- **Security**: Never commit the `.env` file to GitHub!
- **Roles**: There are only two roles: `user` and `admin`. You can manually change a user's role to 'admin' in the MongoDB database if you need admin access locally.

Happy Coding! Hare Krishna! 🙏